Large-scale cluster management at Google with Borg Xiyao Long Response

The paper talks about how Google's Borg system as a cluster manager manages heterogeneous jobs. 'Heterogeneous" here means that the jobs submitted vary in their computational workload and latency-sensitivity. For examples, some of the jobs are end-user faced such as Gmail, Google docs and should guarantee some results within a sub-second timeframe. Some other jobs are less latency sensitive but can be very batch-intensive therefore involves a large number of machines of different physical builds for a single job.

Borg, therefore, needs to handles many of the issues that inevitably arise from such use cases including configuring different machines for running some specific applications and managing their resources for good utilization, resolving machine failure or network problem to ensure reliability and availability.

The basic structure of a Borg job is a set of tasks to be run on a set of machines(Borg cell). User can specify that the machines should have certain attributes. A task can also have resource requirement attached to it. There are other mechanisms such as "Alloc" for reserving resources for future tasks and setting priorities for different tasks. Particularly the tasks priorities setting that corresponding to "production" cannot preempt each other to eliminate preemption cascade. A job also has its set of machines elect a Borgmaster which is a scheduler, handles clientRPCs and communicates with Borglets. The master essentially serves as a Paxos leader and is in charge of submitting a job, terminating tasks. The master can recover itself or restore paste states from saved checkpoint upon invocation by receiving certain RPCs. It records a submitted job and schedule the tasks in the job, and most importantly using a scheduling algorithm to find and score machines that can run an application. For The scoring algorithm, one concern they have is how to make borg handles scheduled tasks across machines to utilize the computing power to a fuller extent. There are some compromises in how tightly they should fit tasks into the capacity of a machine and discussed possible solutions of preempting tasks fo different priorities to resolve load spikes.

Another issue they discussed is how to handle machine failure or in a general sense ensures machine availability. Depending on the real-world scenario, the implemented strategies are auto-rescheduling of failed tasks, running tasks in new machines if the previous machines are unreachable and limiting tasks disruption upon system updating. They claim by using the combination of these techniques they enabled 99/99% availability.

I think they use a Borgmaster which is Paxos based leader for scheduling by itself packages solutions of a lot of cluster management issues already. I'm most interested in that they mark tasks priorities for different bands and enable different strategies for preempting tasks of different bands. I can see this allow finer tasks and resources management to ensure reasonable program/service behavior while also handling load spikes. I think this strategy can be used in other computational tasks where tasks are of different computational requirements and some of them should not be preempted by another subset of the tasks. A flaw in the writing is that they can talk about whether there are any dependencies between the tasks in the job and how they handle them.